

Project 2 Intuition Guide

WaveFEM Solver

Numerical Methods for PDE

Purpose

This document explains what we are going to build before writing the full codebase. It is written for computer science students with weak PDE background.

1 What The Project Is About

We must implement a finite element solver for the 2D wave equation:

$$\frac{\partial^2 u}{\partial t^2} - \Delta u = f \quad \text{in } \Omega$$

with boundary and initial data:

$$u = g \quad \text{on } \partial\Omega, \quad u(0) = u_0, \quad \partial_t u(0) = u_1.$$

Intuitively, $u(x, y, t)$ is a displacement. Think of a vibrating membrane, like a drum surface. At every point (x, y) and time t , u tells how high or low the membrane is.

The project requires both implementation and discussion of space discretization, time discretization, stability, numerical dissipation, numerical dispersion, and computational aspects.

2 Meaning Of The PDE

The equation is

$$u_{tt} - \Delta u = f.$$

The term u_{tt} is acceleration in time. The term $-\Delta u$ describes spatial tension: it compares the value at one point with nearby values. The term f is an external force.

The boundary condition $u = g$ fixes the membrane on the boundary. The easiest first case is $g = 0$, meaning the boundary is fixed at height zero. The initial condition u_0 gives the initial shape, while u_1 gives the initial velocity.

3 Weak Formulation

The computer does not solve the PDE point by point. FEM solves an integrated version. We multiply the PDE by a test function v and integrate:

$$\int_{\Omega} u_{tt} v \, dx - \int_{\Omega} \Delta u \, v \, dx = \int_{\Omega} f v \, dx.$$

After integrating by parts the Laplacian term:

$$\int_{\Omega} u_{tt} v \, dx + \int_{\Omega} \nabla u \cdot \nabla v \, dx = \int_{\Omega} f v \, dx.$$

For homogeneous Dirichlet boundary conditions, the boundary term disappears because the test functions are zero on the boundary.

Final weak form:

$$(u_{tt}, v)_\Omega + (\nabla u, \nabla v)_\Omega = (f, v)_\Omega \quad \forall v \in V.$$

The first term becomes the mass matrix part, the second becomes the stiffness matrix part, and the right-hand side becomes the force vector.

4 From Weak Form To Matrices

In FEM, we approximate

$$u_h(x, y, t) = \sum_j U_j(t) \phi_j(x, y).$$

The functions ϕ_j are basis functions. The unknowns are the coefficients $U_j(t)$. After inserting this approximation into the weak form, we get

$$M\ddot{U}(t) + KU(t) = F(t),$$

where

$$M_{ij} = \int_\Omega \phi_j \phi_i dx, \quad K_{ij} = \int_\Omega \nabla \phi_j \cdot \nabla \phi_i dx, \quad F_i = \int_\Omega f \phi_i dx.$$

This is the key translation from mathematics to code.

5 Time Discretization

The wave equation is second order in time, so we need displacement and velocity, or current and previous displacement. A simple centered finite difference method is

$$M \frac{U^{n+1} - 2U^n + U^{n-1}}{\Delta t^2} + KU^n = F^n.$$

Equivalently,

$$MU^{n+1} = \Delta t^2 F^n - \Delta t^2 KU^n + 2MU^n - MU^{n-1}.$$

This method is intuitive and second order in time, but conditionally stable: Δt must be small enough compared to the mesh size.

Another option is an implicit Newmark method. It is usually more stable for larger time steps, but each time step is more expensive because it requires solving a linear system involving both M and K .

6 Stability, Dissipation, Dispersion

Stability means the numerical solution does not blow up for no physical reason. Numerical dissipation means the method artificially damps the wave. Numerical dispersion means waves travel at wrong numerical speeds, so the shape may shift or distort.

Simple checks:

- with $f = 0$ and $g = 0$, energy should stay roughly constant for non-dissipative methods;
- smaller h and smaller Δt should improve accuracy;
- the wave amplitude should not disappear too quickly unless the method is dissipative;
- the numerical wave should not arrive too early or too late compared to an exact solution.

7 First Validation Test

Use the unit square $\Omega = (0, 1)^2$ and the exact solution

$$u(x, y, t) = \sin(\pi x) \sin(\pi y) \cos(\sqrt{2}\pi t).$$

It satisfies homogeneous Dirichlet conditions because the sine is zero on the square boundary. For this solution:

$$f = 0, \quad u_0(x, y) = \sin(\pi x) \sin(\pi y), \quad u_1(x, y) = 0.$$

This is a good first test because we know what the solution should look like.

8 Implementation Plan

The code should follow the standard deal.II lab style. Suggested structure:

- WaveEquation.hpp: class declaration;
- WaveEquation.cpp: implementation;
- main.cpp: parameters and run call;
- CMakeLists.txt: build configuration;
- README.md: build/run instructions;
- mesh/: optional mesh scripts;
- output/: generated results.

Main methods:

- setup(): create mesh, finite element space, DoFs, matrices, vectors;
- assemble_matrices(): assemble M and K ;
- interpolate_initial_conditions(): fill initial displacement and velocity;
- solve_time_step(): compute the next solution;
- output_results(): write files for ParaView;
- compute_errors(): compare with exact solution when available;
- run(): orchestrate the simulation.

9 Algorithm In Pseudocode

```
read parameters
create mesh
create finite element space
distribute degrees of freedom
assemble mass matrix M
assemble stiffness matrix K
interpolate u0 and u1
compute U^0
compute U^1 or initialize velocity

for each time step:
    assemble or evaluate F(t)
    compute U^{n+1}
```

apply Dirichlet boundary conditions
compute optional error/energy
write output every few steps

write summary CSV files

10 What To Look At In ParaView

The most important visualization is the displacement field u . Useful outputs are: final displacement at time T , animation over time, coarse/fine mesh comparison, and large/small Δt comparison.

Check that the wave moves smoothly, the boundary condition is respected, the solution is not exploding, and the amplitude is plausible.

11 Experiments To Include

Minimum useful experiments:

1. exact solution test on $(0,1)^2$;
2. error vs mesh size h ;
3. error vs time step Δt ;
4. energy over time for the homogeneous case;
5. visual comparison in ParaView.

12 Group Split

One possible split for five people:

- Person 1: weak formulation and report explanation;
- Person 2: deal.II mesh, DoFs, matrices;
- Person 3: time discretization and solver;
- Person 4: validation, errors, energy plots;
- Person 5: ParaView output, README, final presentation.

13 Oral Explanation

Short explanation:

We solve the 2D wave equation with the finite element method in space. After multiplying by test functions and integrating by parts, we obtain a weak form. Using Lagrange basis functions, this becomes a second-order ODE system in time, $MU'' + KU = F$. We then choose a time discretization, apply Dirichlet boundary conditions, and advance the solution step by step. We validate the solver with an exact sinusoidal solution and study stability, dissipation, dispersion, and computational cost.

Even shorter:

FEM turns the PDE into $MU'' + KU = F$. Then the time method turns that ODE system into linear algebra at each time step.

14 Risk Checklist

- sign of the Laplacian term: the weak form should have $+\nabla u \cdot \nabla v$;
- boundary conditions: homogeneous Dirichlet is the easiest first case;
- initial velocity: the wave equation needs both u_0 and u_1 ;
- time step: explicit methods can blow up if Δt is too large;
- output frequency: do not write too many huge files;
- exact solution: verify that f , u_0 , u_1 , and g match the formula;
- convergence: decreasing h and Δt should improve accuracy;
- energy: for the homogeneous case, energy should be approximately conserved unless the method is dissipative.

15 Final Mental Model

The project is a pipeline:

PDE problem $\rightarrow$ weak formulation $\rightarrow$ FEM matrices M, K
 $\rightarrow$ time stepping $\rightarrow$ output $\rightarrow$ validation and report.

If we understand this pipeline, the implementation becomes a structured programming task instead of a mysterious PDE problem.

16 Super Basic Picture

Think of the domain as a flat square surface. At each point, $u(x, y, t)$ is the height of that surface.

- $u = 0$: the surface is flat.
- $u > 0$: the surface is above the flat position.
- $u < 0$: the surface is below the flat position.
- u_0 : the starting shape.
- u_1 : the starting velocity.
- f : an external push.

The solver answers this question: if we know the starting shape and starting speed, what does the surface look like later?

17 Very Small Fake Mesh Example

FEM starts by cutting the domain into small pieces. In a very small square mesh:

```
o----o----o
|    |    |
o----o----o
|    |    |
o----o----o
```

Each `o` is a node. In a simple $P1$ method, the unknowns are the values of u at the nodes. If the boundary is fixed with $u = 0$, then all boundary nodes are known:

```

0----0----0
|      |      |
0----?----0
|      |      |
0----0----0

```

Only the center node is unknown in this tiny example. In a real mesh there are many unknowns, collected in a vector:

$$U = [u_1, u_2, u_3, \dots, u_N].$$

This is why the PDE becomes linear algebra.

18 What The Matrices Mean

The semi-discrete equation is

$$M\ddot{U} + KU = F.$$

Think of it like

$$\text{mass} \cdot \text{acceleration} + \text{stiffness} \cdot \text{displacement} = \text{force}.$$

It is the multi-node version of the simple oscillator equation

$$mx'' + kx = \text{force}.$$

For a toy case with three unknown nodes:

```

U = [height at node 1,
      height at node 2,
      height at node 3]

```

- M tells how node masses/inertia are distributed.
- K tells how each node is connected to nearby nodes by spatial stiffness.
- F tells which nodes are pushed by the source term.

19 One Time Step In Plain Words

At time t_n , assume we know the previous shape U^{n-1} and the current shape U^n . We want the next shape U^{n+1} .

For the centered method:

$$M \frac{U^{n+1} - 2U^n + U^{n-1}}{\Delta t^2} + KU^n = F^n.$$

In code-like form:

```

rhs = dt^2 * F
rhs -= dt^2 * K * current_solution
rhs += 2 * M * current_solution
rhs -= M * old_solution
solve M * next_solution = rhs

```

So each time step builds a right-hand side vector and solves a matrix system.

20 Visual Examples

Flat Start, No Force

If

$$u_0 = 0, \quad u_1 = 0, \quad f = 0,$$

then the solution should remain zero forever. If the code creates waves here, something is wrong.

Initial Bump, No Force

If

$$u_0 = \sin(\pi x) \sin(\pi y), \quad u_1 = 0, \quad f = 0,$$

the membrane starts as one smooth bump, then moves down and up again, oscillating over time. In ParaView this should look like a smooth vibrating surface.

Zero Shape, Initial Velocity

If

$$u_0 = 0, \quad u_1 = \sin(\pi x) \sin(\pi y), \quad f = 0,$$

the membrane starts flat but immediately moves because it has initial velocity. This explains why the wave equation needs both u_0 and u_1 .

External Push

If

$$u_0 = 0, \quad u_1 = 0, \quad f \neq 0,$$

the membrane moves because the source term pushes it.

21 How To Think About ParaView Output

When ParaView opens a `.vtu` or `.pvd` file, we are looking at the vector U drawn on the mesh.

Good signs:

- the boundary stays fixed if $u = 0$ on the boundary;
- the wave is smooth for smooth initial data;
- smaller mesh size gives a cleaner shape;
- smaller time step gives a more reliable animation;
- the solution does not explode.

Bad signs:

- boundary values move even though they should be fixed;
- the solution becomes huge very quickly;
- random oscillations appear;
- the wave disappears too fast without reason;
- output is empty or all one color when it should not be.

22 Minimal Story For The Group

We have a vibrating 2D surface.

We split the surface into a mesh.

The unknown is the height at mesh degrees of freedom.

The weak form gives us two main matrices: M and K.

M handles acceleration in time.

K handles spatial bending/curvature.

The PDE becomes $M U'' + K U = F$.

A time method tells us how to compute the next U.

Then we export U over time and check if the wave behaves correctly.